Document Conversion Service

Complete User Guide

Version 1.0 • 2025

A production-grade microservice that converts documents and images to plain text.
Supports SQS, RabbitMQ, Kafka message buses, REST API, and 10 document formats.

Table of Contents

1. Overview
2. Architecture
3. Prerequisites
4. Quick Start – Docker Compose
5. Configuration Reference
6. Message Schema
7. REST API Reference
8. Using the API Test Console
9. Message Bus Integration
   9.1 Amazon SQS
   9.2 RabbitMQ
   9.3 Apache Kafka
10. Document Converters
11. Image OCR
12. File Fetchers (S3, URL, FTP)
13. Output Storage (S3)
14. Memory Efficiency & Streaming
15. Testing
16. Troubleshooting
17. curl Examples Cookbook

1. Overview

The Document Conversion Service is a containerized Python microservice that accepts conversion jobs from multiple sources – three message buses (Amazon SQS, RabbitMQ, Apache Kafka) and an optional REST API – and converts documents and images into plain text. The extracted text is uploaded to an Amazon S3 bucket (or S3-compatible storage like LocalStack or MinIO).

Every component is independently toggleable via environment variables. You can run the service as a pure queue consumer with no HTTP surface, or as an API-only service, or any combination.

Key Features

- 10 document formats: PDF, DOCX, XLSX, PPTX, HTML, RTF, ODT, TXT, CSV, Image (OCR)
- 3 message buses: SQS, RabbitMQ, Kafka – all using the same JSON message schema
- REST API with file upload and JSON job submission
- Web-based API Test Console (built-in HTML UI)
- 3 document sources: S3, HTTP/HTTPS URLs, FTP servers
- Authentication support: None, Basic, Bearer token
- Memory-efficient streaming via Python generators and temporary files
- PDF image extraction with OCR (Tesseract)
- Full Docker Compose stack with LocalStack, RabbitMQ, Kafka, FTP
- Every component can be enabled/disabled via environment variables

2. Architecture

System Diagram

The service follows a pipeline pattern: Ingest → Fetch → Convert → Upload.

```
┌──────────────────┐   ┌──────────────────┐   ┌──────────────────┐
┌──────────────────┐
│   SQS Queue  │  │  RabbitMQ Q  │  │  Kafka Topic │  │   REST API   │
│ (LocalStack) │  │              │  │              │  │  (FastAPI)   │
└──────────┬───────┘   └──────────┬───────┘   └──────────┬───────┘
└──────────┬───────┘
       │                │                │                │
       └────────────────┴────────────────┴                │
                        ▼                                 ▼
              ┌──────────────────────────────────────────────┐
              │            Conversion Job Router             │
              └──────────────────────┬───────────────────────┘
                                     ▼
              ┌──────────────────────────────────────────┐
              │          Document Fetcher                │
              │  S3  │  HTTP/URL  │  FTP  │  Local      │
              └──────────────────────┬───────────────────┘
                                     ▼  (tmp file on disk)
              ┌──────────────────────────────────────────┐
              │        Converter Dispatcher              │
              │  PDF │ DOCX │ XLSX │ PPTX │ HTML ... │
              └──────────────────────┬───────────────────┘
                                     ▼  (text chunks via yield)
              ┌──────────────────────────────────────────┐
              │      S3 Output Storage                   │
              │  streams text → s3://output-bucket       │
              └──────────────────────────────────────────┘
```

Docker Services

Service	Image	Ports	Purpose
localstack	localstack/localstack:3.4	4566	SQS queues + S3 buckets
rabbitmq	rabbitmq:3.13-management-alpine	5672, 15672	RabbitMQ bus + Management UI
zookeeper	confluentinc/cp-zookeeper:7.6.1	2181	Kafka coordination
kafka	confluentinc/cp-kafka:7.6.1	9092	Kafka message bus
ftp	fauria/vsftpd	21, 21100-21110	Demo FTP server
docconv-app	Built from Dockerfile	8080	The conversion service

Project Structure

```
docconv-service/
├── docker-compose.yml        # Full infrastructure stack
├── Dockerfile                # App container image
├── requirements.txt          # Python dependencies
├── .env.example              # All configurable env vars
├── config/settings.py        # Pydantic settings (single source of truth)
├── app/
│   ├── main.py               # ★ Entry point – enables/disables components
│   ├── models.py             # Pydantic models (ConversionJob, Result)
│   ├── api.py                # FastAPI REST endpoints + test console
│   ├── processor.py          # Pipeline: fetch → convert → upload
│   ├── storage.py            # S3 output uploader
│   ├── static/index.html     # API Test Console (HTML UI)
│   ├── bus/
│   │   ├── sqs_listener.py
│   │   ├── rabbitmq_listener.py
│   │   └── kafka_listener.py
│   ├── fetchers/
│   │   ├── dispatch.py       # Routes by location_type
│   │   ├── s3_fetcher.py
│   │   ├── url_fetcher.py
│   │   └── ftp_fetcher.py
│   └── converters/
│       ├── dispatch.py       # Routes by document_type
│       ├── pdf_converter.py  # PDF text + embedded image OCR
│       ├── docx_converter.py
│       ├── xlsx_converter.py # Also handles CSV
│       ├── pptx_converter.py
│       ├── html_converter.py
│       ├── rtf_converter.py
│       ├── odt_converter.py
│       ├── text_converter.py
│       └── image_converter.py
├── scripts/
│   ├── init-localstack.sh    # Creates SQS queue + S3 buckets
│   └── demo.py               # Send test jobs via every channel
└── tests/
    └── test_full_flow.py     # Integration test suite (41 tests)
```

3. Prerequisites

Requirement	Version	Notes
Docker	20.10+	Docker Desktop or Docker Engine
Docker Compose	v2.0+	Included with Docker Desktop
Disk space	~3 GB	For container images
RAM	4 GB minimum	8 GB recommended for Kafka + LocalStack
Python	3.10+ (optional)	Only needed for running demo.py or tests locally

Note: All service dependencies (Tesseract, Poppler, etc.) are installed inside the Docker image. You do not need to install them on your host.

4. Quick Start – Docker Compose

Step 1: Extract and enter the project

```
unzip docconv-service.zip
cd docconv-service
```

Step 2: Make the init script executable

```
chmod +x scripts/init-localstack.sh
```

Step 3: Build and start all services

```
docker compose up --build -d
```

This builds the app image, pulls infrastructure images, starts all 6 containers, and initializes LocalStack with the SQS queue and S3 buckets.

Step 4: Verify services are healthy

```
docker compose ps
```

All services should show "healthy" or "running" status.

Step 5: Open the API Test Console

Open your browser to http://localhost:8080 to access the built-in API test console.

Step 6: Watch logs

```
docker compose logs -f docconv-app
```

Step 7: Run the demo script (optional)

```
pip install boto3 pika confluent-kafka requests
python scripts/demo.py
```

Step 8: Stop the stack

```
docker compose down          # Stop containers
docker compose down -v       # Stop and remove volumes
```

5. Configuration Reference

All configuration is via environment variables. Every variable has a sensible default. Copy .env.example to .env for local overrides.

Feature Flags

Variable	Default	Description
ENABLE_API	true	Start the FastAPI HTTP server on port 8080
ENABLE_SQS	true	Start the SQS long-poll listener thread
ENABLE_RABBITMQ	true	Start the RabbitMQ consumer thread
ENABLE_KAFKA	true	Start the Kafka consumer thread

Set any flag to "false" to completely disable that component at startup.

SQS Settings

Variable	Default	Description
SQS_ENDPOINT_URL	http://localstack:4566	SQS endpoint (LocalStack or real AWS)
SQS_QUEUE_NAME	docconv-jobs	Name of the SQS queue to poll
SQS_POLL_INTERVAL	5	Long-poll wait time in seconds

S3 Output Settings

Variable	Default	Description
S3_ENDPOINT_URL	http://localstack:4566	S3 endpoint for output uploads
S3_OUTPUT_BUCKET	docconv-output	Default bucket for converted text

RabbitMQ Settings

Variable	Default	Description
RABBITMQ_HOST	rabbitmq	RabbitMQ server hostname
RABBITMQ_PORT	5672	AMQP port
RABBITMQ_USER	docconv	Authentication username
RABBITMQ_PASS	docconv	Authentication password
RABBITMQ_QUEUE	docconv-jobs	Queue name to consume

Kafka Settings

Variable	Default	Description
KAFKA_BOOTSTRAP_SERVERS	kafka:29092	Kafka broker address
KAFKA_TOPIC	docconv-jobs	Topic to subscribe to
KAFKA_GROUP_ID	docconv-group	Consumer group ID

AWS Credentials

Variable	Default	Description
AWS_ACCESS_KEY_ID	test	Used by boto3 for SQS and S3
AWS_SECRET_ACCESS_KEY	test	Used by boto3 for SQS and S3
AWS_DEFAULT_REGION	us-east-1	AWS region

Tuning

Variable	Default	Description
CHUNK_SIZE	65536	Read/write chunk size in bytes (64 KB)
TMP_DIR	/tmp/docconv	Directory for temporary files
LOG_LEVEL	INFO	Python logging level (DEBUG, INFO, WARNING, ERROR)
API_PORT	8080	Port for the FastAPI HTTP server

6. Message Schema

Every message bus and the REST API accept the same JSON schema. Fields are required or optional depending on the location_type.

Full JSON Schema

```
{
  "job_id": "optional-caller-id",
  "document_type": "pdf",
  "location_type": "s3",

  "s3_bucket": "my-bucket",
  "s3_key": "path/to/file.pdf",
  "s3_endpoint_url": null,

  "url": "https://example.com/doc.pdf",

  "ftp_host": "ftp.example.com",
  "ftp_port": 21,
  "ftp_path": "/path/to/file.pdf",
  "ftp_user": "username",
  "ftp_pass": "password",

  "auth_type": "none",
  "auth_username": null,
  "auth_password": null,
  "auth_token": null,

  "output_s3_bucket": null,
  "output_s3_key": null
}
```

Required Fields by Location Type

Field	S3	URL	FTP	Local (API)
document_type	Required	Required	Required	Required
location_type	Required	Required	Required	Set automatically
s3_bucket	Required	—	—	—
s3_key	Required	—	—	—
url	—	Required	—	—
ftp_host	—	—	Required	—
ftp_path	—	—	Required	—

Supported document_type values

Value	Description	Converter Module
pdf	PDF documents (with embedded image OCR)	pdf_converter.py
docx	Microsoft Word documents	docx_converter.py
xlsx	Microsoft Excel spreadsheets	xlsx_converter.py
pptx	Microsoft PowerPoint presentations	pptx_converter.py
html	HTML web pages	html_converter.py
rtf	Rich Text Format	rtf_converter.py
odt	OpenDocument Text (LibreOffice)	odt_converter.py
txt	Plain text (with encoding detection)	text_converter.py
csv	Comma-separated values	xlsx_converter.py
image	Images – JPEG, PNG, TIFF, BMP, WEBP (OCR)	image_converter.py

7. REST API Reference

Base URL: http://localhost:8080 (configurable via API_PORT)

GET /

Returns the API Test Console HTML page. Open in a browser to use the interactive UI.

GET /health

Returns the service status and enabled components.

```
curl http://localhost:8080/health
```

Response:

```
{
  "status": "ok",
  "api_enabled": true,
  "sqs_enabled": true,
  "rabbitmq_enabled": true,
  "kafka_enabled": true
}
```

POST /convert/job

Submit a conversion job. The service fetches the document from the remote source, converts it, and uploads the text to S3.

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{
    "document_type": "html",
    "location_type": "url",
    "url": "https://example.com"
  }'
```

Response:

```
{
  "job_id": "a1b2c3d4e5f6",
  "success": true,
  "output_bucket": "docconv-output",
  "output_key": "converted/a1b2c3d4e5f6.txt",
  "error": null,
  "characters_extracted": 1256
}
```

POST /convert/upload

Upload a file directly for conversion (multipart form data).

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@report.pdf" \
  -F "document_type=pdf"
```

Optional form fields: output_s3_bucket, output_s3_key.

GET /docs

FastAPI auto-generated Swagger/OpenAPI documentation.

Disabling the API

Set ENABLE_API=false to run the service without an HTTP endpoint. The process will still consume from enabled message buses.

8. Using the API Test Console

The service includes a built-in web UI for testing all API endpoints. Open http://localhost:8080 in your browser after starting the Docker stack.

Test Console Features

- Health Check tab – verify connectivity and see which components are enabled
- Upload tab – drag-and-drop or click to upload files for conversion, with auto-detection of document type from the file extension
- Job tab – build and send JSON conversion jobs with S3, URL, or FTP sources, with a live JSON preview panel
- Schema tab – reference for the full message schema and all supported values
- Request history – logs every request with method, path, status code, and response time
- Syntax-highlighted JSON responses with result summary cards

Configuration

The Base URL field at the top defaults to http://localhost:8080. Change it if your service is running on a different host or port. Click 'Test Connection' to verify.

9. Message Bus Integration

The service can consume jobs from SQS, RabbitMQ, and Kafka simultaneously. Each listener runs in its own daemon thread and can be independently enabled or disabled.

9.1 Amazon SQS

The SQS listener long-polls the configured queue. In the Docker Compose stack, LocalStack provides a local SQS emulation. For production, point SQS_ENDPOINT_URL to the real AWS endpoint (or remove it to use the default).

Sending a test message via AWS CLI

```
aws --endpoint-url=http://localhost:4566 sqs send-message \
  --queue-url http://localhost:4566/000000000000/docconv-jobs \
  --message-body '{
    "document_type": "txt",
    "location_type": "s3",
    "s3_bucket": "docconv-input",
    "s3_key": "test.txt"
  }'
```

Sending via Python (boto3)

```
import boto3, json
sqs = boto3.client("sqs",
    endpoint_url="http://localhost:4566",
    aws_access_key_id="test",
    aws_secret_access_key="test",
    region_name="us-east-1")
queue_url = sqs.get_queue_url(QueueName="docconv-jobs")["QueueUrl"]
sqs.send_message(
    QueueUrl=queue_url,
    MessageBody=json.dumps({
        "document_type": "html",
        "location_type": "url",
        "url": "https://example.com"
    }))
```

9.2 RabbitMQ

The RabbitMQ listener connects via AMQP and consumes from the configured queue. The management UI is available at http://localhost:15672 (user: docconv, pass: docconv).

Sending via Python (pika)

```
import pika, json
creds = pika.PlainCredentials("docconv", "docconv")
conn = pika.BlockingConnection(
    pika.ConnectionParameters("localhost", 5672, credentials=creds))
ch = conn.channel()
ch.queue_declare(queue="docconv-jobs", durable=True)
ch.basic_publish(
    exchange="",
    routing_key="docconv-jobs",
    body=json.dumps({"document_type":"pdf","location_type":"url",
                     "url":"https://example.com/doc.pdf"}),
    properties=pika.BasicProperties(delivery_mode=2))
conn.close()
```

9.3 Apache Kafka

The Kafka listener subscribes to the configured topic using the confluent-kafka consumer. Auto-topic creation is enabled in the Docker Compose stack.

Sending via Python (confluent-kafka)

```
from confluent_kafka import Producer
import json
producer = Producer({"bootstrap.servers": "localhost:9092"})
producer.produce("docconv-jobs",
    value=json.dumps({
        "document_type": "docx",
        "location_type": "url",
        "url": "https://example.com/report.docx"
    }).encode("utf-8"))
producer.flush()
```

Sending via Kafka CLI

```
echo '{"document_type":"txt","location_type":"url","url":"https://example.com"}' | \
  docker exec -i docconv-kafka kafka-console-producer \
    --bootstrap-server localhost:29092 \
    --topic docconv-jobs
```

10. Document Converters

Each converter is a standalone Python module exposing a convert_to_text(file_path) function that returns a generator of text chunks. This design keeps memory usage constant regardless of document size.

PDF (pdf_converter.py)

Extracts selectable text via pdfplumber page-by-page. If a page has no selectable text, it falls back to full-page OCR by rendering the page as an image with Poppler's pdf2image and running Tesseract. Additionally, embedded images on every page are extracted and OCR'd separately, with results tagged as [IMAGE TEXT].

DOCX (docx_converter.py)

Reads paragraphs in batches of 50 using python-docx. Also extracts all table content with cell delimiters. Yields text in chunks.

XLSX / CSV (xlsx_converter.py)

For XLSX: reads each sheet with openpyxl in read-only mode, yielding rows in batches of 200 with pipe-delimited cells. For CSV: uses the csv stdlib module with the same batching strategy.

PPTX (pptx_converter.py)

Reads each slide using python-pptx. Extracts text from all shapes (text frames and tables) plus speaker notes. Yields one slide at a time.

HTML (html_converter.py)

Strips script/style/noscript tags using BeautifulSoup + lxml. Extracts visible text with newline separators. Yields in configurable chunk sizes.

RTF (rtf_converter.py)

Strips RTF control words using the striprtf library. Yields plain text in chunks.

ODT (odt_converter.py)

Reads OpenDocument Text paragraphs via odfpy. Yields in batches of 50 paragraphs.

TXT (text_converter.py)

Detects encoding via chardet, then reads and yields text in CHUNK_SIZE chunks. Handles UTF-8, Latin-1, Windows-1252, and other encodings.

11. Image OCR

The image converter (image_converter.py) handles JPEG, PNG, TIFF (including multi-frame), BMP, and WEBP formats. It uses Pillow to open the image and Tesseract for OCR.

Processing Steps

- Open the image with Pillow
- For multi-frame TIFFs, iterate through each frame
- Down-scale if either dimension exceeds 4000px (prevents OOM)
- Convert to RGB if needed (handles RGBA, palette modes)
- Run Tesseract OCR and yield the extracted text

PDF Image Extraction

The PDF converter additionally extracts images embedded in each page using pdfplumber's image detection. Each image is cropped from the page, rendered at 200 DPI, and OCR'd. Results are tagged with [IMAGE TEXT] prefix.

12. File Fetchers (S3, URL, FTP)

Each fetcher downloads a document to a local temporary file using chunked streaming. All fetchers use Python generators with yield to ensure the tmp file is cleaned up after processing, even if an error occurs.

S3 Fetcher (s3_fetcher.py)

- Uses boto3 to get the S3 object
- Reads the response body in CHUNK_SIZE chunks
- Writes to a NamedTemporaryFile in TMP_DIR
- Supports custom endpoint URLs for LocalStack/MinIO

URL Fetcher (url_fetcher.py)

- Uses requests with stream=True for chunked download
- Supports auth_type=none, basic (username/password), and bearer (token)
- Timeout set to 120 seconds

FTP Fetcher (ftp_fetcher.py)

- Uses stdlib ftplib with retrbinary for binary download
- Supports anonymous and authenticated access
- Reads in CHUNK_SIZE blocks

13. Output Storage (S3)

Converted text is uploaded to the S3 output bucket. The upload process writes yielded text chunks to a local tmp file first, then uploads the complete file to S3. This approach avoids buffering large text strings in memory and handles S3 upload retries gracefully.

Default Output Path

```
s3://{S3_OUTPUT_BUCKET}/converted/{job_id}.txt
```

Override with output_s3_bucket and output_s3_key in the job message.

Checking Output

```
# List converted files
aws --endpoint-url=http://localhost:4566 s3 ls \
  s3://docconv-output/converted/ --recursive
```

```
# View a converted file
aws --endpoint-url=http://localhost:4566 s3 cp \
  s3://docconv-output/converted/a1b2c3d4e5f6.txt -
```

14. Memory Efficiency & Streaming

The service is designed for low memory usage when processing large documents.

- **Fetchers:** Fetchers:
- Stream remote files to disk in CHUNK_SIZE chunks using Python generators (yield). The file is written to TMP_DIR and auto-deleted after processing.
- **Converters:** Converters:
- Yield text one page/chunk at a time. They never build the full text string in memory. PDF pages are processed and discarded individually.
- **Upload:** Upload:
- Writes yielded chunks to a tmp file, then uploads that file to S3. No full-text buffering in RAM.
- **PDF images:** PDF images:
- Rendered to tmp PNG files for OCR, deleted immediately after each page is processed.
- **API uploads:** API uploads:
- Streamed to disk in chunks from the HTTP request body, never held in memory.

15. Testing

Running the Integration Test Suite

```
cd docconv-service
pip install -r requirements.txt reportlab httpx
python -m tests.test_full_flow
```

What the Tests Cover (41 tests)

Section	Count	Description
Sample Generation	10	Creates sample files for every supported format
Converter Unit Tests	10	Tests each converter module independently
Dispatcher Tests	10	Verifies DocumentType → converter routing
Pipeline Tests	10	End-to-end: file → convert → output
Model Validation	4	Pydantic schema validation and rejection
REST API Tests	7	Health, upload (5 formats), validation

Running via Docker

```
docker compose exec docconv-app python -m tests.test_full_flow
```

16. Troubleshooting

Services fail to start

Run "docker compose logs <service>" to check for errors. Ensure ports 4566, 5672, 9092, 8080 are not in use. Increase Docker memory to 8 GB if Kafka/Zookeeper OOM.

SQS listener says 'queue not found'

LocalStack may still be initializing. The listener retries 30 times with 2-second intervals. Check that init-localstack.sh is executable (chmod +x).

OCR returns empty text

Ensure the tesseract-ocr and tesseract-ocr-eng packages are installed in the Docker image (they are by default). Check the image quality – very low resolution or handwritten text may produce poor results.

PDF conversion is slow

Full-page OCR (for scanned PDFs) renders each page at 200 DPI. This is CPU-intensive. For faster processing of selectable-text PDFs, no OCR is triggered.

Upload fails with 'Connection refused'

The S3 upload is targeting LocalStack. Ensure LocalStack is running and healthy. Check with: curl http://localhost:4566/_localstack/health

RabbitMQ connection refused

The listener auto-retries every 5 seconds. RabbitMQ may take 10-15 seconds to start. Check: docker compose logs rabbitmq

Kafka consumer not receiving messages

Ensure the topic exists or auto-creation is enabled (KAFKA_AUTO_CREATE_TOPICS_ENABLE=true). Check bootstrap server address matches between producer and consumer.

API Test Console shows 'Disconnected'

Click 'Test Connection' to retry. Ensure the Base URL matches your Docker host. If running Docker in a VM, use the VM's IP instead of localhost.

17. curl Examples Cookbook

Health check

```
curl http://localhost:8080/health
```

Upload a PDF

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@document.pdf" \
  -F "document_type=pdf"
```

Upload an image for OCR

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@scan.png" \
  -F "document_type=image"
```

Upload a DOCX with custom output key

```
curl -X POST http://localhost:8080/convert/upload \
  -F "file=@report.docx" \
  -F "document_type=docx" \
  -F "output_s3_key=reports/2025/q1.txt"
```

Convert from URL

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"html","location_type":"url",
       "url":"https://example.com"}'
```

Convert from URL with Basic auth

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"pdf","location_type":"url",
       "url":"https://secure.example.com/doc.pdf",
       "auth_type":"basic",
       "auth_username":"user","auth_password":"pass"}'
```

Convert from URL with Bearer token

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"xlsx","location_type":"url",
```

```
       "url":"https://api.example.com/export.xlsx",
       "auth_type":"bearer","auth_token":"tok_abc123"}'
```

Convert from S3 (LocalStack)

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"pdf","location_type":"s3",
       "s3_bucket":"docconv-input","s3_key":"docs/report.pdf",
       "s3_endpoint_url":"http://localstack:4566"}'
```

Convert from FTP

```
curl -X POST http://localhost:8080/convert/job \
  -H "Content-Type: application/json" \
  -d '{"document_type":"txt","location_type":"ftp",
       "ftp_host":"ftp","ftp_path":"/data/file.txt",
       "ftp_user":"docconv","ftp_pass":"docconv"}'
```

List converted output files

```
aws --endpoint-url=http://localhost:4566 s3 ls \
  s3://docconv-output/converted/ --recursive
```

Download and view converted text

```
aws --endpoint-url=http://localhost:4566 s3 cp \
  s3://docconv-output/converted/JOB_ID.txt -
```